

Lecture 09: LC-3 Microcode and Branches

Date: Tuesday, June 16th, 2026

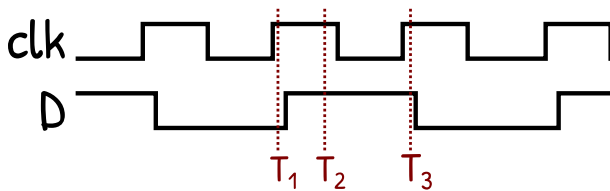
Reading: Patt & Patel 3rd ed.: Sec. 5.4.3; Sec. 5.4.1

Announcements:

- HW3 due yesterday (grace period is right now)
 - Q2.1 multiple answers
- HW4 out, due Saturday 11:59pm
- Project 1 due Wednesday 11:59pm
- Project 2 out tomorrow, maybe Thurs
- Quiz 2 topic list out later today
 - Content: Lecture 4 part 2 (sequential logic) through this lecture
 - Reference sheet

Part 0: Timing Diagrams for Flip-Flops

A student on Ed asked how to approach finding the value of Q (the output of a flip flop) at the points T_1 , T_2 , and T_3 in a timing diagram such as the following:

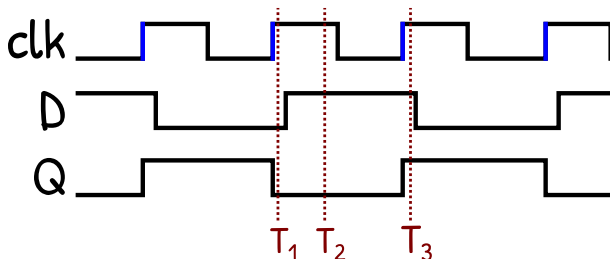


First, as an analogy, imagine a homework problem in a Python class that asked you to find the value of `sum` when `i==5` in the following code:

```
sum = 0
for i in range(10):
    sum = 2*sum + i
```

You would *not* approach this problem by considering the iteration `i==5` in isolation, scratching your head until you had a giga-brain moment that delivers you the value of `sum`. Instead, you would *trace through the program*, keeping track of the current value of `sum` during simulated execution, until you hit `i==5`. We will follow the same idea here: we will trace how the flip flop changes its value over time starting at the beginning of the timing diagram.

Suppose the quiz question stated, “the initial value held in the flip-flop is 0. Find Q at times T_1 , T_2 , and T_3 ”. Then we (the student) would begin by drawing how Q , the current value of the flip-flop, changes over time from left to right as follows:



As long as we were careful in drawing the Q signal, the answer is right there in the diagram:

- At time T_1 , $Q = 0$
- At time T_2 , $Q = 0$
- At time T_3 , $Q = 1$

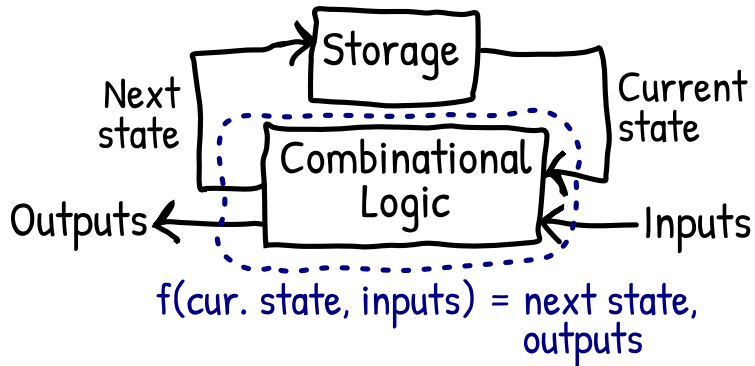
Now, we will discuss new material.

Part 1: Microcode

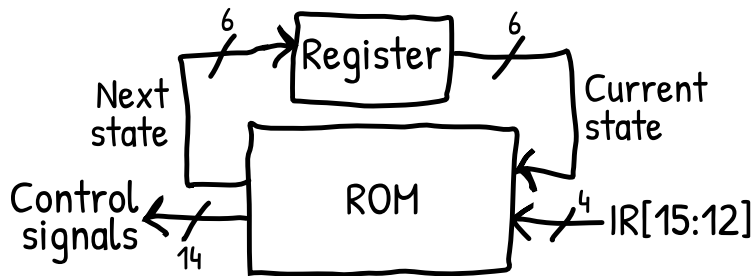
Learning objective for Lecture 09 Part 1:

- Course learning objective #3: **Analyze** the structure and **examine** the operation of a basic von Neumann CPU.
 - **Convert** a state transition diagram describing the LC-3 FSM to microcode (a table of bits).

Recall the framework for sequential logic from Lecture 05 Part 1:



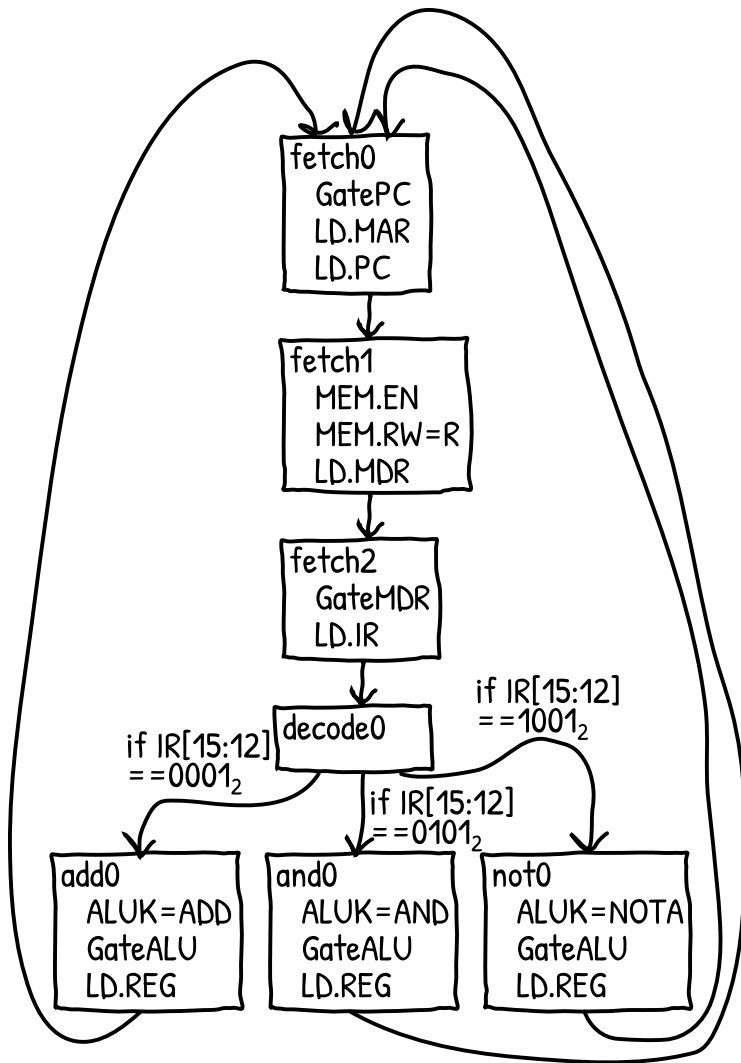
Following this framework, the FSM in the control unit of our LC-3 microarchitecture is implemented as the following circuit:



Choosing 6 bits for the state register is determined by the number of states we need, which happens to be around 55 per Appendix C of the textbook.

Here, **ROM** stands for **Read-Only Memory**. Concatenating the current state bits $S_5S_4S_3S_2S_1S_0$ with the FSM input $IR[15:12]$ forms an address. Looking up this address in the ROM yields the control signals (outputs) and the next state number.

How does this ROM look for the following state transition diagram, for instance?



You can imagine this ROM as a giant truth table for a function (f above) that chooses the control signals and next state given a current state and opcode ($IR[15:12]$). In the representation of this table written below, the first column is a convenient name of the state for humans (e.g., *fetch0*) that does not exist physically but corresponds to some choice of state bits $S_5S_4S_3S_2S_1S_0$. The next three columns $S_2S_1S_0$ (the state number) are the **address** of that row, and the remaining columns are outputs and next state bits, the **value** at that row. (To fit in the page, this example only shows the lower three state bits, only the lower three next state bits, and a **subset** of control signals. For the same reason, the $IR[15:12]$ input is also omitted.)

	S_2	S_1	S_0	LD.MAR	LD.MDR	LD.PC	LD.IR	GatePC	N_2	N_1	N_0
<i>fetch0</i>	0	0	0	1	0	1	0	1	0	0	1
<i>fetch1</i>	0	0	1	0	1	0	0	0	0	1	0
<i>fetch2</i>	0	1	0	0	0	0	1	0	0	1	1

Decode works by using the $IR[15:12]$ input to choose the new state bits (below, only ADD, AND, and NOT are handled as an example, except their execution control signals are omitted in the interest of page space):

	S_2	S_1	S_0	$IR[15]$	$IR[14]$	$IR[13]$	$IR[12]$	N_2	N_1	N_0
<i>decode0</i>	0	1	1	0	0	0	1	1	0	0
<i>decode0</i>	0	1	1	0	1	0	1	1	0	1
<i>decode0</i>	0	1	1	1	0	0	1	1	1	0
<i>add0</i>	1	0	0	x	x	x	x	0	0	0
<i>and0</i>	1	0	1	x	x	x	x	0	0	0

<i>not0</i>	1	1	0	x	x	x	x	0	0	0
-------------	---	---	---	---	---	---	---	---	---	---

(Writing don't cares, the xs, as inputs as I have above is shorthand for duplicating that row for all possible combinations of bits in the x positions.)

Note that *add0*, *and0*, and *not0* return back to *fetch0* once they're finished executing.

This table of bits is called **microcode**. You will write it yourself in a big Excel spreadsheet in Project 2. On Quiz 2, you will also be expected to convert a state transition diagram to microcode. The second page of the LC-3 reference sheet shows our arbitrary mapping of bits to control signal named values.

With this necessary implementation information dealt with, we can continue to extending to LC-3 ISA and our microarchitecture to support conditionals and loops.

Part 2: Branch Instructions

Learning objectives for Lecture 09 Part 2:

- Course learning objective #3: **Analyze** the structure and **examine** the operation of a basic von Neumann CPU.
 - **Write** the state transition diagram for executing branch instructions (JMP/BR).
- Course learning objective #4: **Identify** the structure and demonstrate the function of machine language instructions
 - **Interpret** 16 bits as an LC-3 branch instruction (JMP/BR).
 - **Simulate** LC-3 branch instructions (JMP/BR).
- Course learning objective #5: **Assemble** a symbolic assembly language and **describe** its structure and purpose.
 - **Assemble** LC-3 assembly syntax for branch instructions (JMP/BR) into bits.

Let's check the status of our ISA to-do list:

To-do list to get a useful ISA

- | |
|---|
| <ul style="list-style-type: none"> • ☒ Arithmetic/logical instructions • ☒ Load/store instructions • ☒ Branch instructions (change PC) |
|---|

In this lecture, we will discuss LC-3 branch instructions, which enable control flow such as if-else statements, loops, and function calls.

The JMP Instruction

A *branch* instruction is a kind of instruction that updates the PC during the Execute phase. Because the PC holds the address of the next instruction to execute, branch instructions achieve control flow.

The first idea for implementing a branch instruction is to set the PC to the value held in a temporary (R0-R7). This is what JMP does.

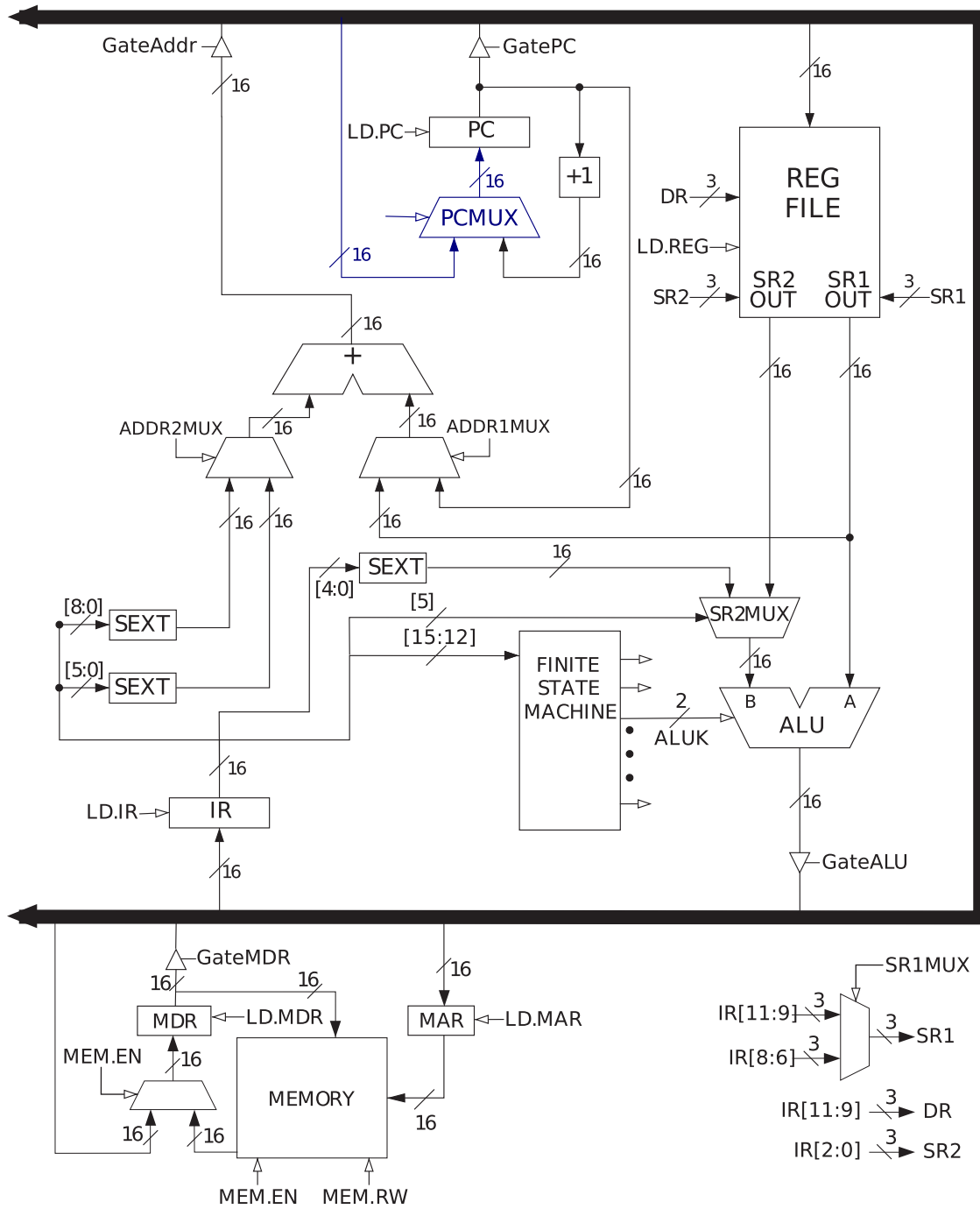
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JMP	1	1	0	0	0	0	0	BaseR			0	0	0	0	0	0

The semantics (behavior) for JMP are the following:

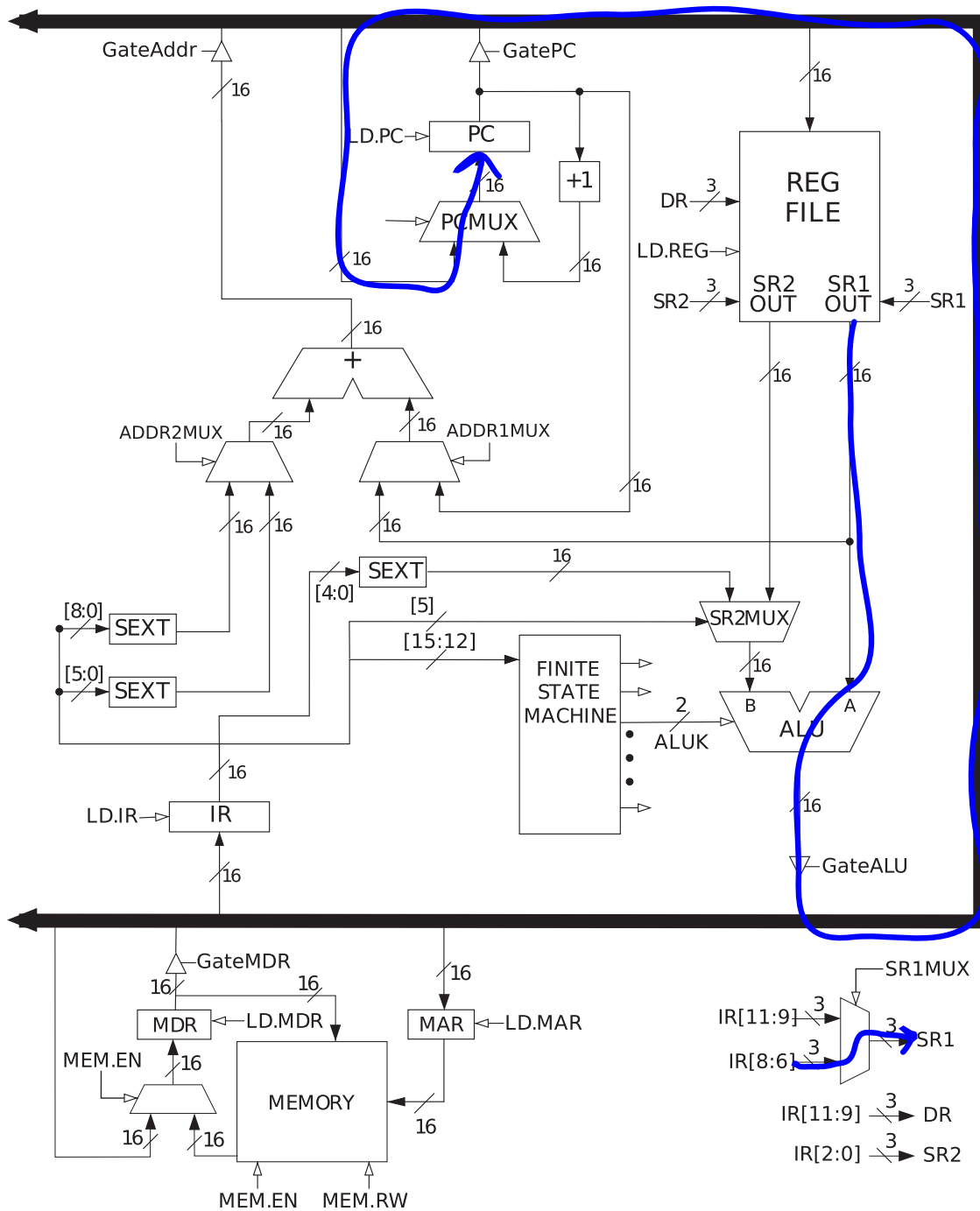
$$PC \leftarrow \text{BaseR}$$

The assembly syntax for JMP looks like JMP BaseR, e.g., JMP R3.

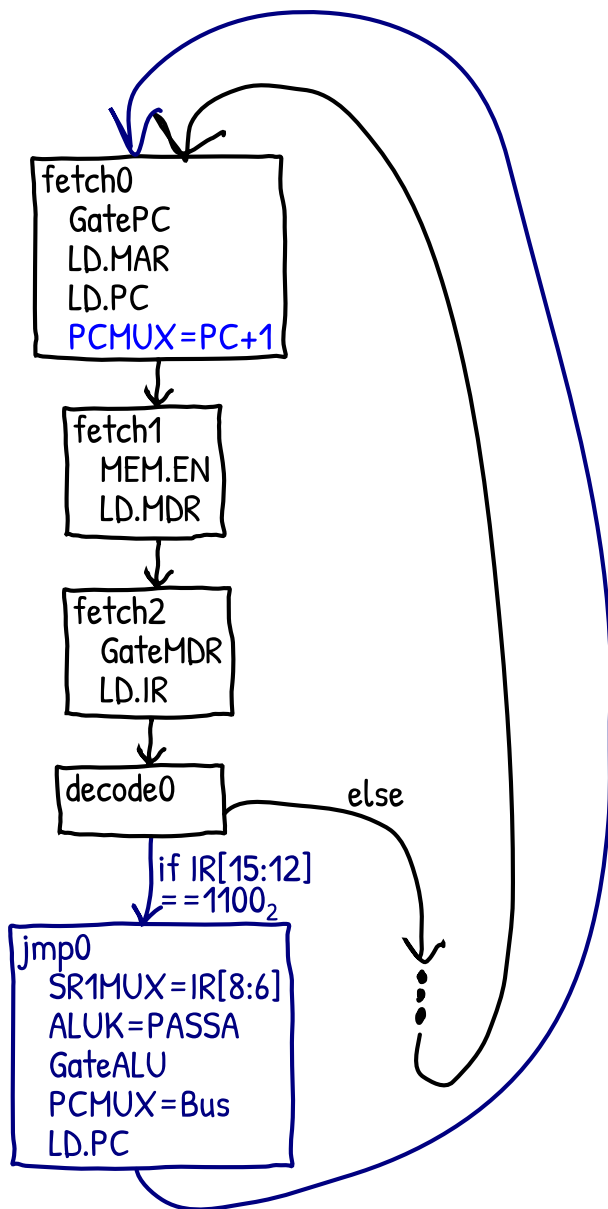
To implement the JMP instruction in the LC-3 microarchitecture, we need to add a way to change the PC:



This way, a value can flow from the register value into the PC:



This flow can be achieved by adding the following new state (`jmp0`) to the FSM:

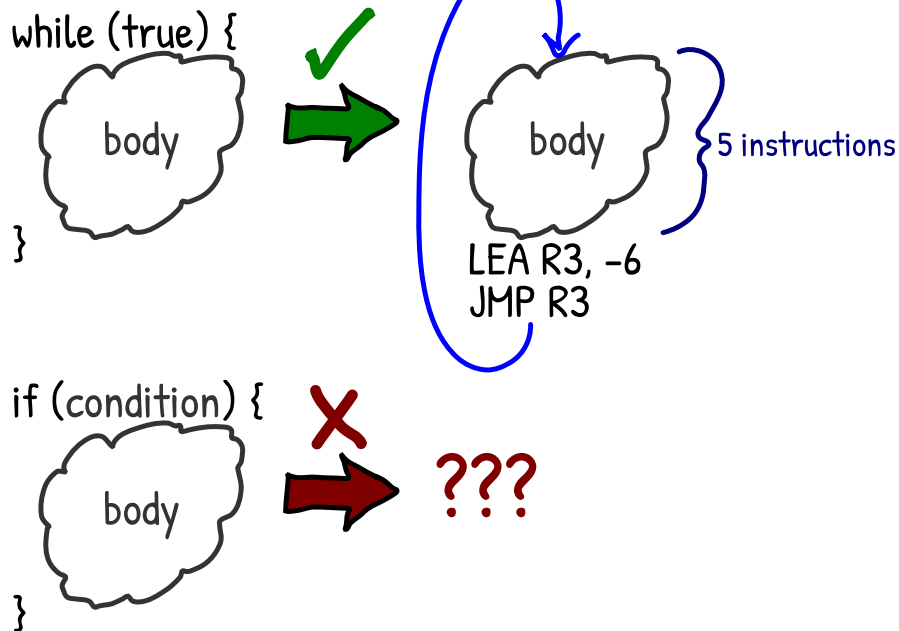


Note that we also updated fetch0 to set PCMUX correctly. As in prior state transition diagrams we've drawn for the FSM in our LC-3 microarchitecture, "PC+1" and "Bus" above are shorthand for the selector bits that makes the mux output the corresponding choice. In a concrete circuit that implements this FSM, PC+1 and Bus would actually be replaced by specific distinct bits, but which bits those happen to be is an implementation detail. In this class, we are computer architects, not digital design engineers!

The JMP instruction allows us to implement an infinite loop as follows:

High-level Language

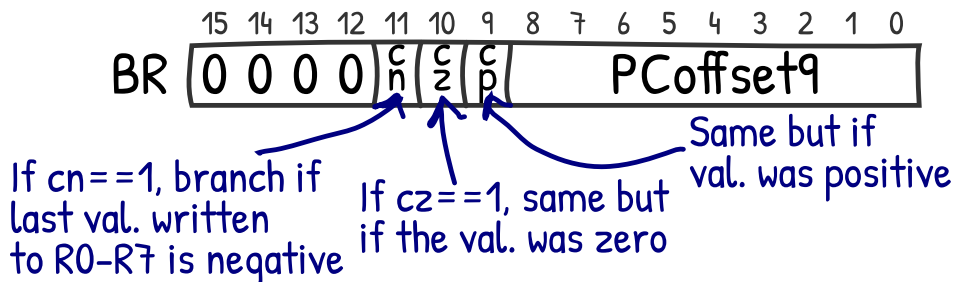
LC-3 Instructions



But wait, what about if statements? We cannot conveniently implement an if statement because JMP is an **unconditional branch** (an instruction that always changes the PC), but we need a **conditional branch** (an instruction that changes the PC only if a condition is met). The next section will introduce the conditional branch instruction in the LC-3 ISA.

The BR Instruction

In the LC-3 ISA, the BR instruction is encoded as follows:



The semantics of BR are the following:

$$\text{if } ((\text{cn AND } N) \text{ OR } (\text{cz AND } Z) \text{ OR } (\text{cp AND } P)) \{ \\ \text{PC} \leftarrow \text{PC}^* + \text{SEXT}(\text{PCoffset9}) \\ \}$$

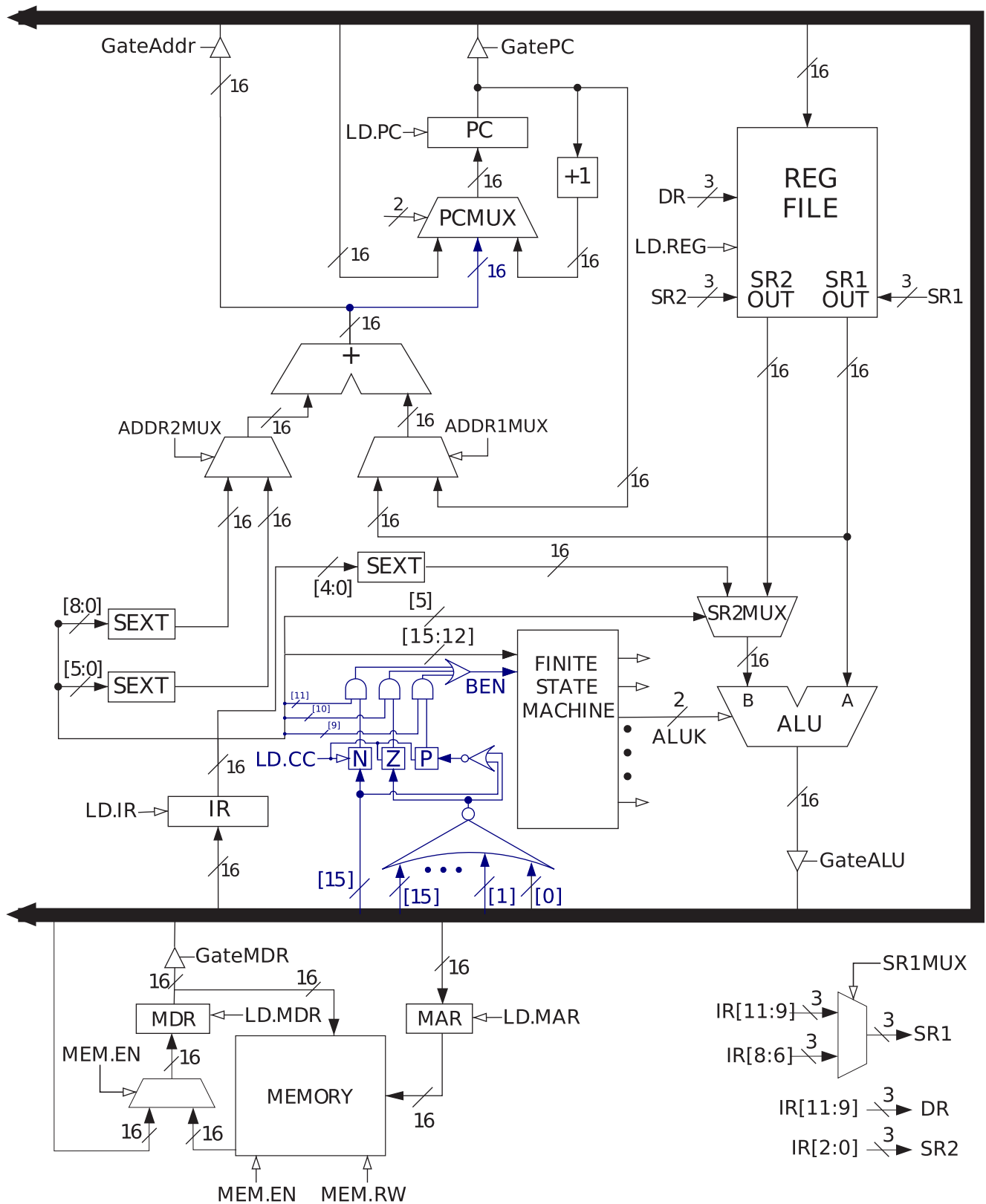
In other words, if the condition is met, the PC offset is added to the PC. Else, the PC is left alone, meaning execution continues with the next instruction following the BR in memory. But what are the N , Z , and P used in the condition $(\text{cn AND } N) \text{ OR } (\text{cz AND } Z) \text{ OR } (\text{cp AND } P)$? They (N , Z , and P) are 1-bit registers on the datapath, each of which hold 1 if and only if the last value written to R0-R7 was negative, zero, or positive, respectively. Together, N , Z , and P are known as the **condition code (CC)**.

The assembly syntax for BR is unlike any other LC-3 instruction in that cn, cz, and cp are determined by changing the mnemonic:

- The mnemonic BRn means $cn = 1, cz = 0, cp = 0$
- The mnemonic BRz means $cn = 0, cz = 1, cp = 0$
- The mnemonic BRp means $cn = 0, cz = 0, cp = 1$
- The mnemonic BRnp means $cn = 1, cz = 0, cp = 1$
- The mnemonic BRzp means $cn = 0, cz = 1, cp = 1$
- The mnemonic BRnz means $cn = 1, cz = 1, cp = 0$
- The mnemonic BRnzp *or* BR means $cn = 1, cz = 1, cp = 1$
- The mnemonic NOP (short for *no-op*) means $cn = 0, cz = 0, cp = 0$
 - You cannot pass a PC offset to this form, since it would never be used

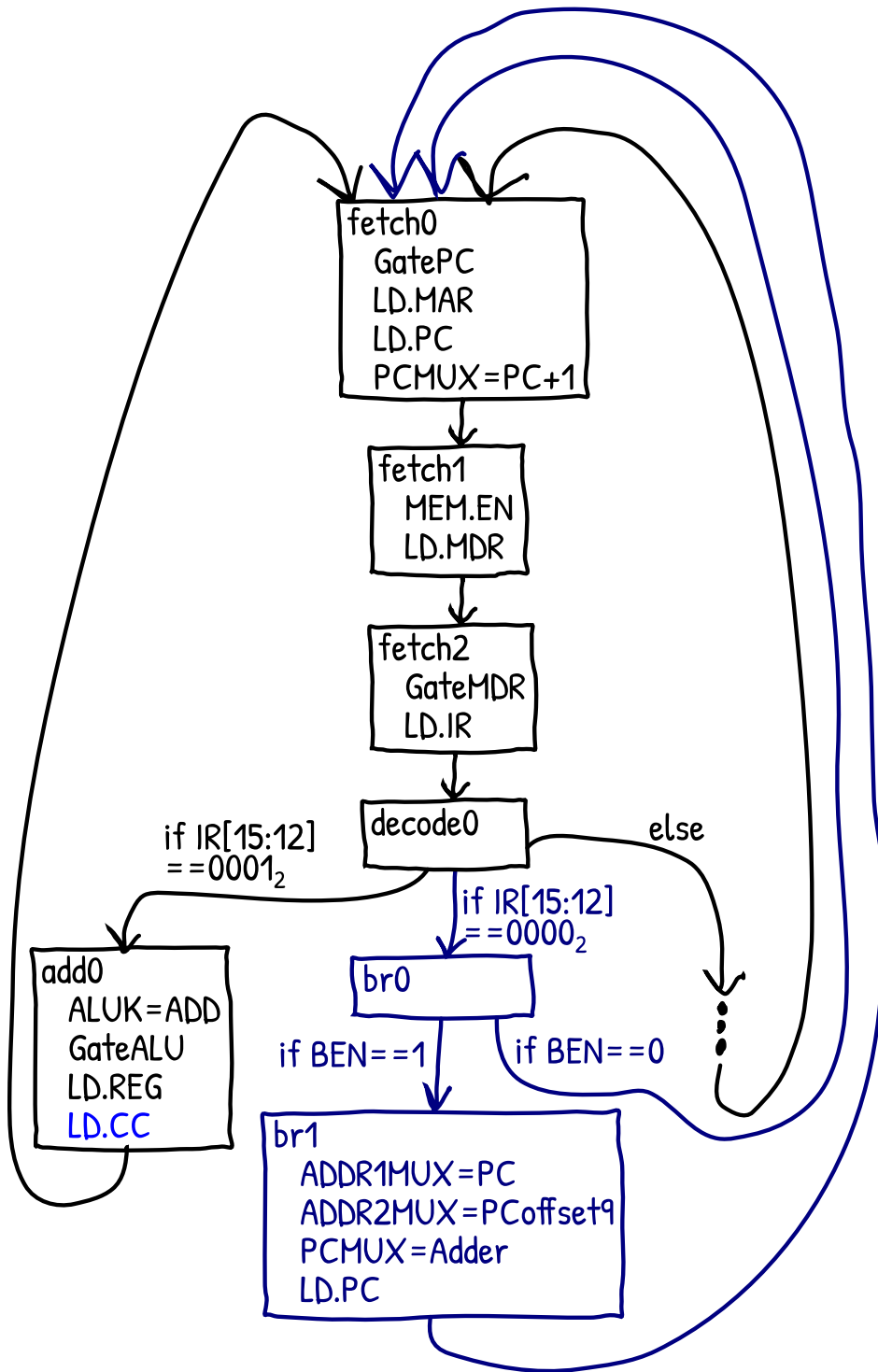
Example: BRzp -3 adds -3 to the PC if the CC indicates that the last value written to R0-R7 is either zero or positive. Else, the PC is left alone.

The CC is added to the datapath as follows:



The circuitry between the three CC registers (*N*, *Z*, *P*) and the bus determines whether the value on the bus is negative, zero, or positive according to the rules of 16-bit two's complement. (Henceforth, this circuitry will be drawn as a box labeled "LOGIC" to keep the datapath from getting too crowded.) The write enable port of all three registers is connected to the **LD.CC** signal. The FSM must assert **LD.CC** whenever a value is being written to a temporary *R0-R7*, since the value to be written will be on the bus (more on this later).

To support BR in the FSM for our LC-3 microarchitecture, we can make the following modifications:



I am hoping that we have now gained enough experience with control signals that the incorporation of the `br0` and `br1` states feels natural.

Note that we changed the existing `add0` state to assert `LD.CC`. This is because `add0` writes to `R0-R7`, and we promised to update the `CC` whenever we write to `R0-R7`. Although it is not shown in this picture, the `not0` and `and0` states must have this modification as well. In fact, any state that updates registers must assert `LD.CC` alongside `LD.REG`; this includes states for executing `LEA`, `LD`, `LDI`, `LDR`, etc. in which registers are modified.

Conclusion

Let's revisit our to-do list:

To-do list to get a useful ISA

- ☒ Arithmetic/logical instructions
 - ☒ Load/store instructions
 - ☒ Branch instructions (change PC)
-

At this point, having implemented functionally complete arithmetic/logic instructions, various load/store instructions, and both conditional and unconditional branches in the LC-3 ISA, our toy microarchitecture has grown into a capable computer. But right now, programming it by writing bits by hand is agonizing. Next lecture we will address this using LC-3 assembly language.